DEMYSTIFYING INTELLIGENT AUTOMATION

A Beginner's Guide to RPA & AI

Hands-On Lab Manual • 6 Guided Projects on the Free UiPath Community Edition

One-Day Industry Workshop for Engineering Students

Morning: Concepts & Live Demo | Afternoon: Hands-On Lab (2–3 hrs)

Prepared for	Engineering College Workshop Audience (Mixed Programming Background)
Facilitator	[Add your name / organization here]
Software	UiPath Studio (Community Edition, free) + UiPath Automation Cloud (free tier) + UiPath Studio Web
Optional add-on	Free Google AI Studio (Gemini) API key for Projects 5 & 6 — no credit card required

Table of Contents

Table of Contents 2
How to Use This Manual 4
Suggested Tracks 4
Prerequisites & Setup Checklist 4
1. Accounts 4
2. Software (for desktop projects — 1, 2, 3) 4
3. Browser-only requirements (for Studio Web — 4, 5, 6) 5
4. Sample files — included with this manual 5
Workshop Projects at a Glance 5
Part 1 — Before You Begin: UiPath & RPA Fundamentals 7
1.1 — Creating Your Account and Installing Studio 7
1.2 — Your First Look at Studio 7
1.3 — Variables and Data Types 8
1.4 — The Assign Activity and Writing Expressions 9
1.5 — The If Activity (Making Decisions) 10
1.6 — Loops: For Each and For Each Row 11
1.7 — Installing Extra Activity Packages (Manage Packages) 11
1.8 — Just Enough Regular Expressions (Regex) 12
1.9 — Running and Debugging Your Bot 13
1.10 — Studio Web in a Nutshell 14
1.11 — Where Are My Sample Files? 16
Automated Excel Data Cleanup & Report 17
Learning Outcomes 17
Step-by-Step Instructions 17
Expected Output 19
Web Data Scraper to Excel 21
Learning Outcomes 21
Step-by-Step Instructions 21
Expected Output 22
PDF Invoice Extraction Bot 24
Learning Outcomes 24
Step-by-Step Instructions 24
Expected Output 26
Automating a Web Form with Studio Web 27
Learning Outcomes 27

Step-by-Step Instructions 27
Expected Output 28
AI-Powered Customer Feedback Analyzer 30
Learning Outcomes 30
Step-by-Step Instructions 30
Before You Start: Is This Actually Free on a Community License? 30
Expected Output 34
Autonomous Support Ticket Triage Agent 35
Learning Outcomes 35
Step-by-Step Instructions 35
Expected Output 38
Appendix A: UiPath Activity Cheat Sheet 39
Appendix B: GPT Prompt Templates for RPA 39
Classification (single label) 39
Structured extraction (JSON) 39
Summarization 39
Draft reply generation 39
Appendix C: Glossary 40
Appendix D: A Note on API Details 40
Appendix E: Where to Find the Sample Files 40

(If this section looks empty, right-click it and choose "Update Field" — Word usually does this automatically on open.)

How to Use This Manual

This manual assumes you have never used UiPath, and never written a line of code. That's the whole point of Part 1 (Fundamentals) — read it before touching Project 1, since every project's steps refer back to it instead of re-explaining variables, loops, or expressions each time.

Every project is self-contained after that: read the objective, follow the numbered steps and lettered micro-steps, and check your work against the Expected Output section before moving on.

Suggested Tracks

Track	Projects	Focus
Foundation (brand new to coding)	Project 1 → Project 2	Studio basics: sequences, variables, Excel, web scraping
Builder (some coding experience)	Project 2 → Project 3 → Project 4	Desktop automation + first Studio Web flow
Advanced (comfortable with logic/coding)	Project 4 → Project 5 → Project 6	Studio Web + AI-powered "agentic" automation

> **TIP**
> Don't rush. A working 3-step bot you understand is worth more than a 10-step bot you copied without understanding. Ask a neighbor or a facilitator before you get stuck for more than 5 minutes — and re-read the relevant section of Part 1 first, since most "stuck" moments trace back to one of the concepts explained there.

Prerequisites & Setup Checklist

Complete this checklist before the workshop begins — ideally the night before, since installs and account approvals can take time.

1. Accounts

- Create a free UiPath Automation Cloud account at cloud.uipath.com (this also gives access to UiPath Studio Web and UiPath Academy). Full click-by-click steps are in Part 1, §1.1.
- Verify your email — UiPath sends a verification link.
- A Google account (personal Gmail works) — needed for both Google Sheets (Projects 4–6 data) and a free Google AI Studio API key (Projects 5–6).
- (Projects 5 & 6 only) Create a free Google AI Studio API key at aistudio.google.com — no credit card required. Full steps are at the start of Project 5.

2. Software (for desktop projects — 1, 2, 3)

- Install UiPath Studio — Community Edition (free). See Part 1, §1.1 for the exact download path.
- Install Microsoft Excel or have access to a compatible spreadsheet app (LibreOffice Calc also works for most steps).

- Install Google Chrome or Microsoft Edge with the UiPath browser extension enabled (Studio will prompt you on first web automation).

3. Browser-only requirements (for Studio Web — 4, 5, 6)

- A modern browser (Chrome/Edge), logged into your Automation Cloud account. No local install is required for these projects.
- For Projects 5 and 6: your data is connected via Google Sheets rather than a direct file upload — see Project 5's setup steps for exactly why and how.

4. Sample files — included with this manual

- This manual ships with a Workshop_Sample_Files folder. Copy it to your Desktop or Documents before starting. It contains:
  - Student_Marks_Raw.xlsx — for Project 1
  - Invoices/ (5 sample PDF invoices) — for Project 3
  - Applicants.csv — for Project 4
  - Reviews.csv — for Project 5
  - Tickets.csv — for Project 6
  - (Project 2 needs no file — it pulls live data from a public practice website.)

> **⚠TROUBLESHOOTING**
> Corporate or college lab machines sometimes block software installers or browser extensions. If Studio won't install, use the Foundation/Builder track projects (1–2) on a personal laptop, or switch entirely to the Studio Web track (4–6), which needs only a browser.

Workshop Projects at a Glance

#	Project	Platform	Difficulty	Time
1	Automated Excel Data Cleanup & Report	Studio (Desktop)	Beginner	45–60 min
2	Web Data Scraper to Excel	Studio (Desktop)	Beginner–Intermediate	45–55 min
3	PDF Invoice Extraction Bot	Studio (Desktop)	Intermediate	55–70 min
4	Automating a Web Form with Studio Web	Studio Web	Beginner	40 min
5	AI-Powered Feedback Analyzer	Studio Web + Gemini API	Intermediate	60–75 min
6	Autonomous Ticket Triage Agent	Studio Web + Gemini API	Advanced (Capstone)	75–90 min

Projects 5 and 6 are the AI/GPT-style exercises — they show students how to call a large language model from inside a UiPath workflow and use its output to drive automated decisions, using Google's free-tier Gemini API so no paid account is required (see the feasibility note at the start of Project 5). This pattern — RPA plus LLM reasoning — is often called agentic automation.

> **TIP**
>
> These time estimates assume the reader is following Part 1 (Fundamentals) as reference while working, not re-learning it from scratch mid-project. A full run of every project back-to-back is closer to a full day than a single afternoon — most groups should expect to complete 2–3 projects in a 2–3 hour lab, per their chosen track. Projects 5 and 6 in particular now include a required Delay between every API call to respect the free tier's rate limit, which stretches their run time — see each project's steps for why.

Part 1 — Before You Begin: UiPath & RPA Fundamentals

This chapter assumes zero prior experience with programming or with UiPath. Read it once, fully, before starting Project 1. Every project later in this manual refers back to the concepts explained here (for example, "as covered in §1.5") instead of re-explaining them — so this chapter is the key that unlocks all six projects.

> **TIP**
> This is a reading chapter, not a doing chapter. You don't need Studio open right now — just read and absorb the vocabulary. You will practice everything hands-on starting in Project 1.

1.1 — Creating Your Account and Installing Studio

Step 1. Create your Automation Cloud account. Go to cloud.uipath.com in your browser. Click Sign Up (top right). Enter your email, create a password, and click Create Account. Check your inbox for a verification email and click the verification link.

Step 2. Log in. Return to cloud.uipath.com and log in with the email and password you just created. You will land on the Automation Cloud home page — this is your control center for everything UiPath.

Step 3. Download UiPath Studio. On the left-hand navigation menu, look for Admin (a gear/shield icon). Click it, then click on your tenant name (usually "DefaultTenant"), then find a Downloads or Resources section. Click Download next to "UiPath Studio". If you can't find this path, search the page for a "Downloads" link — UiPath periodically moves this menu item.

Step 4. Run the installer. Once downloaded, double-click the installer file (UiPathStudioSetup.exe on Windows). Click Next through the prompts, accept the license agreement, and click Install. This takes 5–10 minutes.

Step 5. Sign in inside Studio. When Studio opens for the first time, it will ask you to sign in. Choose Automation Cloud, and log in with the same email/password from Step 1. When asked to pick a license, choose Community License (this is completely free — do not enter any payment details).

> **⚠TROUBLESHOOTING**
> If your college computer won't let you install software (common on shared lab machines), skip straight to the Studio Web projects (Project 4, 5, 6) later in this manual — those need only a browser, no installation.

1.2 — Your First Look at Studio

Step 1. Open Studio. Double-click the UiPath Studio icon on your desktop (or find it in your Start Menu).

Step 2. Create a new project. On the Studio Start screen, click Process (a large tile, usually top-left). A dialog box appears asking for a Name, Location, and Description.

a) In the Name field, type a short name with no spaces, e.g. MyFirstBot.

b) Leave Location as the default folder, or click the … (three dots) button to choose a different folder.

c) Click Create (bottom right of the dialog).

Step 3. Studio now opens your empty project. You'll see several distinct areas on screen — this layout never changes, so learning it now saves you time on every project. Refer to this table whenever you feel lost:

Area	Where it is	What it's for
Activities Panel	Far left, with a search box at the top	A searchable library of every building block ("activity") you can drag onto your workflow. Type here to find one, e.g. type "assign".
Design Canvas	Large empty area in the center	Where you build your automation by dragging activities from the Activities Panel and dropping them here, top to bottom.
Properties Panel	Far right (press F4 if it's hidden)	Shows the settings for whichever activity you currently have selected on the canvas. Every activity has different properties.
Variables Panel	Bottom of the screen, a tab labelled "Variables"	Lists every named piece of data ("variable") your workflow currently stores, and lets you create new ones manually.
Project Panel	Far left, a tab next to Activities	Shows all the files in your project, similar to a folder view.
Output Panel	Bottom of the screen, a tab labelled "Output"	Shows text messages while your bot runs — critical for troubleshooting (you will use Write Line activities to print messages here).
Design Ribbon	Top of the window	Contains buttons like Run, Debug, and Manage Packages.

Step 4. Try adding your first activity. In the Activities Panel search box, type Sequence. Drag the "Sequence" activity onto the empty canvas and drop it — a Sequence is simply a container that runs its contents from top to bottom, one after another. Almost everything you build in this manual lives inside a Sequence.

> **TIP**
> Dragging = click and hold the activity in the left panel, move your mouse onto the canvas without releasing, then let go ("drop") when you see a green outline showing where it will land.

1.3 — Variables and Data Types

> **WHAT IS "VARIABLE"?**
> A named box that holds one piece of information while your bot runs — like a labeled sticky note. Instead of writing the number 40 everywhere in your workflow, you store it once in a variable (e.g. passingMarks) and refer to that name.

Every variable has a Type, which tells Studio what kind of data it's allowed to hold. The types you'll meet in this manual:

Type	Holds	Example
String	Text	"Aditi Sharma"
Int32	A whole number	40
Double	A decimal number	1250.50
Boolean	True or False only	True
DataTable	An entire spreadsheet-like grid of rows & columns in memory	the contents of an Excel sheet

Type	Holds	Example
List(Of String)	A growable list of text items	["Rohan", "Karan"]

Two ways to create a variable

1. The fast way: click inside any activity's input or output box, type a name for your new variable (e.g. dtRaw), then press Ctrl+K. Studio automatically creates the variable and guesses its type from context.
2. The manual way: click the Variables tab at the bottom of the screen, click Create Variable, type a name in the Name column, then click the Variable type dropdown and pick a type. If the type you need isn't listed (like List(Of String)), click Browse for Types… and search for it.

> **TIP**
> If you're ever unsure which way to use: use Ctrl+K while you're actively configuring an activity (it's faster), and use the Variables panel when you want to see or rename everything you've already created.

1.4 — The Assign Activity and Writing Expressions

> **WHAT IS "ASSIGN ACTIVITY"?**
> The single most-used activity in RPA. It stores a value into a variable. Search "Assign" in the Activities Panel and drag it onto your canvas — it appears as one row with a box on the left, an equals sign, and a box on the right.

Step 1. The left box ("To"). Type the name of the variable that will receive the value, e.g. passCount.

Step 2. The right box ("Value"). Type an expression — a small formula that Studio calculates. This is written in a language called VB.NET. You don't need to "learn programming" to use it here; you only need to recognize a handful of patterns, all listed below.

Expression patterns used in this manual

Plain text	`"Hello"  — always wrap text in double quotes`
A number	`40  — no quotes needed for numbers`
Join text together	`firstName & " " & lastName   (the & symbol glues text pieces together)`
Add 1 to a counter	`passCount = passCount + 1`
Read a spreadsheet cell	`row("Marks")   — reads the value in the "Marks" column of the current row`
Convert text to a whole number	`CInt(row("Marks").ToString)   — CInt = "Convert to Integer"`
Convert a value to text	`someVariable.ToString`
Compare values	`CInt(row("Marks").ToString) < 40   — is it less than 40?`
Add an item to a list	`failList.Add(row("StudentName").ToString)`

> ⚠ **TROUBLESHOOTING**
> The single most common beginner mistake: forgetting quotation marks

around text, or adding quotation marks around a number/variable by mistake. "Marks" (with quotes) is a literal column name; Marks (without quotes) would mean "a variable called Marks", which doesn't exist and will show a red error.

A tiny worked example (try this before Project 1)

Let's build a 2-activity "bot" that adds two numbers, just to see the pattern before you touch real data:

a) Drag a Sequence onto a new blank workflow.

b) Inside it, drag an Assign activity. In the To box type total, press Ctrl+K to create the variable (Studio will ask the type — choose Int32). In the Value box type 5.

c) Drag a second Assign activity below it. In the To box type total again (same variable). In the Value box type total + 10.

d) Drag a Message Box activity (search for it) below that. In its Text field type "Total is: " & total.ToString.

e) Press F5 (or click the Run button) to run the workflow. A pop-up should appear saying "Total is: 15".

TIP
If that pop-up appeared correctly, you now understand the exact mechanism used in every Assign activity in this manual — only the variable names and values change.

1.5 — The If Activity (Making Decisions)

WHAT IS "IF ACTIVITY"?
Runs one set of activities when a condition is True, and a different set when it's False. Search "If" in the Activities Panel and drag it onto the canvas.

Step 1. Set the Condition. Click the If activity to select it, then look at the top of its box on the canvas — there's a Condition field. Type a comparison expression here, e.g. CInt(row("Marks").ToString) < 40. This must evaluate to True or False, never to a number or text directly.

Step 2. Fill in Then. Click inside the box labelled Then (it usually says "Drop activity here"). Drag whatever activities should run when the condition is True into this box.

Step 3. Fill in Else. Do the same in the Else box for when the condition is False. The Else box is optional — you can leave it empty if you don't need it.

1.6 — Loops: For Each and For Each Row

> **WHAT IS "LOOP"?**
> An activity that repeats a block of steps once per item in a collection (a list, a table's rows, or a folder's files) — instead of you copy-pasting the same steps 20 times, the loop runs them 20 times automatically.

For Each Row (loops over the rows of a DataTable — used with Excel data)

Step 1. Add the activity. Type "For Each Row" in the Activities Panel search box. Drag it onto the canvas.

Step 2. Set what it loops over. Click the small text next to "in" and type the name of your DataTable variable, e.g. dtClean.

Step 3. Use the current row inside the loop. By default each cycle of the loop gives you a variable called row representing the current row. Drag any activities that should repeat (e.g. an If activity) into the loop's body — the empty box below the header.

For Each (loops over a plain list — used with files or lists of text)

Step 1. Add the activity. Type "For Each" in the Activities Panel search box (note: without "Row"). Drag it onto the canvas.

Step 2. Set the collection. In the small field next to "in", type the collection to loop over — for example Directory.GetFiles("Invoices", "*.pdf") to loop over every PDF file in a folder named Invoices.

Step 3. Set the TypeArgument. Click the small down-arrow / "..." near the top-right corner of the For Each box to open its properties, and make sure TypeArgument is set to String (since a list of file paths is a list of text).

Step 4. Use the current item. Each cycle gives you a variable called item (you can rename it in the For Each box) holding the current file path or list entry.

1.7 — Installing Extra Activity Packages (Manage Packages)

Studio starts with a base set of activities (Excel, basic UI automation, etc.), but some activities — like the PDF ones used in Project 3 — live in separate, optional packages you install once per project.

Step 1. Open the Manage Packages window. Click the Design tab at the top of Studio (if it isn't already selected), then click the Manage Packages button (it looks like a small box/cube icon).

Step 2. Go to the right tab. In the window that opens, make sure the left-hand list shows All Packages or Official — this is Studio's default source of activity packages published by UiPath.

Step 3. Search for the package. In the search box at the top, type the package name, e.g. UiPath.PDF.Activities.

Step 4. Install it. Click on the package in the results list, then click the Install button that appears on the right-hand details pane. Click Save when prompted. Wait for the small progress spinner to finish — Studio is downloading and adding the package to your project.

Step 5. Confirm. Close the Manage Packages window. Search the Activities Panel for "Read PDF Text" — if it now appears, the package installed successfully.

> **TIP**

You only need to install a package once per project (not once per activity). If you start a brand-new project later, you'll need to install it again for that new project.

1.8 — Just Enough Regular Expressions (Regex)

WHAT IS "REGULAR EXPRESSION (REGEX)"?

A tiny pattern-matching language for finding specific pieces of text inside a bigger block of text — e.g. finding just the invoice number inside a whole page of invoice text. It looks cryptic at first, but this manual only ever uses a handful of symbols.

Symbol	Means	Example
\d	Any single digit (0-9)	\d\d\d matches "123"
\s	Any whitespace (space, tab, newline)	matches the gap between words
\w	Any letter, digit, or underscore	\w+ matches "INV1001"
+	One or more of the previous symbol	\d+ matches "1", "12", "12345"...
*	Zero or more of the previous symbol	\s* matches any amount of spacing, including none
( )	A "capturing group" — the part you actually want to extract	Total:\s*\$([\d,]+\.\d{2}) captures just the number after $
?	Makes the previous symbol optional	#? matches an optional # character

In this manual, you'll write a line like this to extract a value:

```
invoiceNo = System.Text.RegularExpressions.Regex.Match(pdfText, "Invoice
#:\s*(\S+)").Groups(1).Value
```

Read it in three parts: (1) Regex.Match(textToSearch, pattern) searches pdfText for the pattern; (2) the pattern "Invoice #:\s*(\S+)" looks for the literal text "Invoice #:", then any spacing, then captures the next non-space chunk (\S+ means "one or more non-space characters"); (3) .Groups(1).Value pulls out just the part inside the parentheses — the captured invoice number itself, not the whole matched phrase.

1.9 — Running and Debugging Your Bot

Button / Key	What it does
Run (F5)	Runs your whole workflow from start to finish at full speed.
Debug File (F6)	Runs your workflow but stops at any breakpoints you've set, so you can inspect variables.
Step Into (F11)	Runs just ONE activity at a time — the best tool when something isn't working and you want to watch it happen slowly.
Breakpoint	Click in the thin strip on the left edge of an activity to place a red dot — Debug will pause right before that activity runs.
Locals panel	Appears at the bottom during debugging; shows the current value of every variable in scope, so you can see exactly what your bot "sees".

TIP

When something doesn't work, don't guess — press F6 and F11 to step through slowly and watch the Locals panel. 90% of beginner bugs are visible the moment you actually look at what a variable contains at that

Button / Key	What it does
point in the run.	

1.10 — Studio Web in a Nutshell

Studio Web (used in Projects 4, 5, and 6) is UiPath's browser-based builder. The concepts above (variables, loops, If, expressions) are identical — only the vocabulary and visuals differ slightly:

Desktop Studio term	Studio Web equivalent
Activity	Card
Activities Panel	A "+" button between cards, opening a card picker
Workflow (.xaml file)	Automation
Properties Panel	Click directly on a card to expand its settings inline
Run (F5)	A Run button, top-right of the screen

1.10a — The Two Field Modes: Plain Text vs. Expression Editor

> ⚠TROUBLESHOOTING
> This is the single biggest source of confusion in Studio Web, discovered the hard way while building Projects 5 and 6 — read this carefully before you start either one.

A text field on a Studio Web card can behave in one of two completely different ways, and the card usually doesn't tell you which mode it's in:

Mode	What you type	How to tell / switch to it
Plain text + chip	Ordinary text, exactly as it will be sent — no VB.NET syntax, no & concatenation. To insert a variable's value, you click an @ icon (or similar) and pick from a list, which inserts a visual "chip" at your cursor.	The default on most simple fields. If you type VB.NET-style code here, it gets sent completely literally, character for character — producing confusing, invalid results.
Full expression (code)	Real VB.NET code: string.Format(...), indexers like table("col")(0), .ToString, etc. — exactly like a desktop Studio expression field.	Look for a small icon at the corner of the field — often three lines / "..." / a {} symbol — labelled Expression Editor. Click it to switch the whole field into code mode.

The proven, reliable pattern for this manual: switch the field to Expression Editor mode and build the whole JSON body with a single string.Format call, rather than typing plain text plus a chip. This is what a real, working project actually uses:

```
string.Format("{{""contents"": [{{""parts"": [{{""text"": ""Your prompt text here:
{0}""}}]}}]}}", currentDataRow1.ColumnName)
```

Read this the same way as the desktop Studio patterns in §1.4: string.Format(template, value) drops value into the {0} placeholder inside template. The doubled curly braces {{ and }} are needed because string.Format treats single braces as placeholder markers, so a literal { or } in your JSON must be written doubled. The doubled quotes "" work exactly as in desktop Studio (§1.4) — two double-quotes in a row inside a quoted string produce one literal double-quote character.

> **⚠TROUBLESHOOTING**
> A real bug found in a student's live project: typing an expression like ticketData("category").ToString() but leaving the WHOLE thing wrapped in an extra pair of quotes — "ticketData(""category"").ToString()" — silently turns working code into a literal piece of text. The variable then holds the words "ticketData("category").ToString()" instead of the actual category. If a Switch or If using that variable always falls through to the same branch no matter what, check for exactly this: an accidental extra pair of quotes wrapped around the entire expression.

1.10b — The Hidden Timeout on HTTP Request Cards (and the Retry That's Already Built In)

> **⚠TROUBLESHOOTING**
> If you've hit repeated "A task was canceled" errors calling an AI API from Studio Web, this is almost certainly why.

The HTTP Request card has a Timeout property that isn't shown on the main card face — it defaults to 10,000 milliseconds (10 seconds) whether or not you ever set it. A short, simple prompt (like "reply with one word") often finishes inside 10 seconds, but a longer or more complex prompt asking a "thinking" model to reason through a classification task can easily take longer — and when it does, the call gets silently cancelled with exactly the generic "A task was canceled" error seen throughout this manual's troubleshooting sections, with no indication that a timeout was the cause.

Step 1. Find it. Click the HTTP Request card. Look for an Advanced section, a "Show more" toggle, or scroll the settings panel — Timeout (sometimes shown in milliseconds) is usually tucked away here rather than on the main visible fields.

Step 2. Raise it. Set it to at least 30000 (30 seconds); 60000 (60 seconds) is safer for longer classification-style prompts.

> **TIP**
> The confirmed, working projects this manual is based on actually left this at its 10-second default throughout, and mostly ran fine — the "A task was canceled" errors that came up during testing turned out to be a mix of this timeout and genuine, temporary slowdowns on Google's side (a 503 "high demand" response). Treat raising this as a targeted fix for when you actually hit the error, not a mandatory first step.

Separately, the HTTP Request card already retries automatically on common transient failures — 408, 429, 500, 502, 503, and 504 status codes — up to 3 times with an increasing delay between attempts, without you needing to configure anything. This means the 429 ("rate limit exceeded") and 503 ("model overloaded") errors covered later in this manual are already partly handled for you; the Delay card taught alongside each HTTP call is still worth keeping to avoid hitting the rate limit in the first place, since relying on retries alone means work still gets duplicated and slows down.

1.10c — Reading Data Back Out: Range Field, Row Access, and Column Names

Three details about the Read Range (Google Workspace) card that aren't obvious from the field labels alone:

- The Range field takes just the sheet/tab name (e.g. Reviews, Tickets, Applicants) — not a cell range like Sheet1!A1:B11. Whatever your Google Sheet's tab is named at the bottom, type that exact name here.
- Always turn on HasHeaders — this tells the card to treat your first row as column names rather than data, and is what makes dot-notation access (below) work at all.
- Each row from a For Each looping over readRange.Rows.AsEnumerable is accessed with dot notation matching your column headers exactly — e.g. currentDataRow1.ReviewText, currentDataRow1.Message — not the row("ColumnName") bracket syntax used in desktop Studio DataTables (§1.4). Both exist in UiPath, but Studio Web's Google Sheets connector produces the dot-notation kind.

1.11 — Where Are My Sample Files?

Every dataset used in this manual (the Excel workbook, CSV files, and sample invoice PDFs) is provided alongside this manual in a folder or zip file called Workshop_Sample_Files. Copy that whole folder to your computer before starting — for example, to your Desktop or Documents folder — and note down the path, since every project tells you exactly which file to open from it.

> **TIP**
> For Projects 5 and 6 specifically: rather than uploading the CSV directly, you'll import it into Google Sheets first and connect Studio Web to that sheet. Full steps are at the start of Project 5 — this workaround exists because Studio Web's data-import options turned out to differ from a simple "upload a file" button.

PROJECT 1

Automated Excel Data Cleanup & Report

Your first real bot: read messy data, clean it, and generate a summary. Uses Assign, If, and For Each Row — covered in §1.3–1.6.

Platform	UiPath Studio (Desktop)
Difficulty	Beginner
Est. Time	45–60 minutes
Prerequisites	UiPath Studio installed (§1.1). Sample file Student_Marks_Raw.xlsx from the Workshop_Sample_Files folder (§1.11).

Learning Outcomes

- Create a new Studio project and add Excel activities to it.
- Read a spreadsheet into a DataTable and loop through every row.
- Use variables, an Assign activity, and an If activity together to make a decision per row.
- Write a summary back into a new sheet.

Step-by-Step Instructions

Step 1. Create a new project. Open Studio. On the Start screen click Process. Name it CleanMarksReport, leave the location as default, click Create.

Step 2. Copy the sample file into your project folder. Find Student_Marks_Raw.xlsx inside the Workshop_Sample_Files folder shared with this manual. Copy it into the same folder as your new project (Studio shows this folder path at the top of the Project panel on the left) — this keeps everything in one place.

Step 3. Add a Sequence. In the Activities Panel (left side, search box at top), type Sequence. Drag it onto the empty Design Canvas and drop it. Double-click it to open it (or it may already be open) — you'll build everything inside this Sequence.

Step 4. Add an Excel scope. In the Activities Panel search box, type Excel. You'll see an activity called Use Excel File. Drag it inside the Sequence.

Step 5. Point it at your file. Click on the Use Excel File activity to select it. Look at the Properties Panel on the right (press F4 if you don't see it). Find the property called WorkbookPath and click the folder icon next to it, then browse to and select your copy of Student_Marks_Raw.xlsx.

Step 6. Read the data into a DataTable. Inside the Use Excel File activity's body (a box that says "Drop activities here"), search the Activities Panel for Read Range and drag it in.

a) Click the Read Range activity. In its properties (right panel), find SheetName and type RawData (matching the sheet name inside the sample file).

b) Find the Output property called DataTable. Click inside that box, type dtRaw, and press Ctrl+K. A small dialog may ask you to confirm the type — choose DataTable if asked. This creates your dtRaw variable (see §1.3).

Step 7. Preview what you just built. Click Run (top ribbon, or press F5). If no red error icons appear on your activities, the file was read successfully.

Step 8. Create a variable to count passes. Click the Variables tab at the bottom of the screen. Click Create Variable. In the Name column type passCount. Click the Variable type dropdown and choose Int32. In the Default column type 0 (this makes sure it starts at zero instead of empty).

Step 9. Create a variable to list failing students. Still in the Variables panel, click Create Variable again. Name it failList. Click the Variable type dropdown, click Browse for Types…, type List, and choose List<T> then set the type inside the angle brackets to String (the panel will show it as List(Of String)). This is exactly the List(Of String) type described in §1.3.

Step 10. Add the loop. Back on the canvas, below your Read Range (but still inside the Excel scope's Sequence — drop it right under Read Range), search the Activities Panel for For Each Row and drag it in.

a) Click on the For Each Row activity. Next to the word "in", type dtRaw (the DataTable you created in Step 6).

b) Notice the small text "row" shown at the top of the loop body — that's the automatic variable representing the current row on every cycle, as explained in §1.6.

Step 11. Skip blank rows first. Inside the For Each Row body, drag an If activity (search "If"). Set its Condition to:

```
String.IsNullOrWhiteSpace(row("StudentName").ToString) = False
```

a) This reads as: "the StudentName cell is NOT blank" — so we only process real students.

b) Leave Else empty. Everything else in this project goes inside the Then branch of this If.

Step 12. Add a second, nested If to check the marks — this is the step many beginners find tricky, so we go field-by-field. Inside the Then branch of the If from Step 11, drag another If activity.

a) Click inside its Condition field (the single-line box at the top of the If activity on the canvas).

b) Type exactly: CInt(row("Marks").ToString) < 40

c) Read this left to right: row("Marks") reads the value in the Marks column for the current row. .ToString converts whatever is there into plain text. CInt(…) then converts that text into a whole number (Convert to Integer) so it can be compared mathematically. Finally < 40 checks whether that number is less than 40.

d) This will only show an error if you mistype a quotation mark or a bracket — double-check you have exactly two double-quotes around Marks and that every (has a matching).

Step 13. Configure the Then branch (marks below 40 — a fail). Click inside the Then box of this new If. Drag an Assign activity in.

To	`failList`
Value	`failList.Add(row("StudentName").ToString)`

a) row("StudentName").ToString reads the name as text; .Add(…) is a method that appends an item onto the end of a list — this is the ".Add() method": it simply means "put this new item at the end of failList".

Step 14. Configure the Else branch (marks 40 or above — a pass). Click inside the Else box of the same If. Drag an Assign activity in.

To	`passCount`
Value	`passCount + 1`

a) This says: "take the current value of passCount, add 1 to it, and store the result back into passCount." The variable appears on both sides of the equals sign on purpose — that's the standard pattern for a running counter.

Step 15. Leave the Excel scope and write the summary. Click on empty canvas space to make sure you're no longer nested inside the loop. Below the entire For Each Row activity (but still inside the Use Excel File scope from Step 4, so the workbook is still open), drag a Write Cell activity.

SheetName	`"Summary"`
Range	`"A1"`
Value	`"Pass Count: " & passCount.ToString`

a) SheetName "Summary" — since this sheet doesn't exist yet in the workbook, Write Cell will create it automatically.

b) The Value expression joins the label text with the number using &, after converting the number to text with .ToString (you cannot join text and a number directly with & without converting first).

Step 16. Add one more Write Cell for the fail list. Drag a second Write Cell activity right below the first one.

SheetName	`"Summary"`
Range	`"A2"`
Value	`"Below 40: " & String.Join(", ", failList)`

a) String.Join(", ", failList) turns your whole failList into one piece of text, with each name separated by a comma and space — much easier than looping through the list manually to build the text yourself.

Step 17. Run the whole thing. Click Run (F5). If any activity shows a red exclamation mark, click on it — the error message at the bottom tells you exactly which line and property has the problem (usually a typo in an expression).

Step 18. Check your output. Open Student_Marks_Raw.xlsx yourself in Excel. You should now see a new tab called Summary with the pass count and the list of failing student names.

Expected Output

A new "Summary" sheet inside Student_Marks_Raw.xlsx showing the total number of passing students and the names of every student scoring below 40, with blank rows correctly skipped.

⚠TROUBLESHOOTING

"Object reference not set" on Read Range almost always means the SheetName you typed doesn't exactly match the sheet's real name — open the Excel file and check the tab name at the bottom (case-sensitive and space-sensitive). If CInt throws a conversion error, a Marks cell contains something non-numeric (like spaces) — this is exactly why Step 11's blank-check exists; if it still fails, add Trim() to remove stray spaces: CInt(row("Marks").ToString.Trim()).

⍰ EXTENSION CHALLENGE

Add a bar chart to the Summary sheet using the Excel activity Insert Chart, or wrap Step 12's threshold (40) in its own variable named passingMarks so a teacher could change the cutoff without editing the expression.

PROJECT 2

Web Data Scraper to Excel

Extract structured data from a real website using a wizard, then filter and save it — reinforces expressions and loops from §1.4 and §1.6.

Platform	UiPath Studio (Desktop)
Difficulty	Beginner–Intermediate
Est. Time	45–55 minutes
Prerequisites	UiPath Studio + browser extension enabled. No sample file needed — you'll pull live data from the public practice site books.toscrape.com, built specifically for scraping practice.

Learning Outcomes

- Understand what a selector is and how UiPath finds elements on a web page.
- Use the Extract Data wizard to pull a repeating table of information from a live website.
- Filter the extracted data reliably and write it to Excel.
- Handle multi-page (paginated) data automatically.

Step-by-Step Instructions

Step 1. **Create a new project.** Studio Start screen → Process → name it BookScraper → Create.

Step 2. **Add a Sequence, then open a browser.** Drag a Sequence onto the canvas. Inside it, search the Activities Panel for Use Application/Browser and drag it in.

a) A small setup wizard may appear asking you to indicate a window — click Browser, then in the Url or address field that appears, type https://books.toscrape.com/ directly instead of picking a live window.

b) If prompted to install a browser extension, click Yes/Install and follow the one-time browser prompt — this lets UiPath talk to your browser.

Step 3. **Launch the Extract Data wizard.** With the browser activity's body open (click Run once if needed to actually open the page), go to the Design tab in the top ribbon and click Extract Data.

a) Your live browser window should open showing books.toscrape.com. Click the title of the first book on the page.

b) Studio highlights similar elements and asks you to click a second, different book title to confirm the pattern — click any other book title.

c) A preview table appears. Click the "+" or "Extract Correlated Data" option (wording varies by version) to also capture the Price for each book — click on one price, then confirm.

d) If asked "extract data from multiple pages?", click Yes, then click the "next" link at the bottom of the page when prompted, so Studio learns how to move to page 2 automatically.

e) Click Finish. Studio inserts a ready-made block of activities into your workflow automatically, ending in a DataTable output — click that output field and rename it to bookData (Ctrl+K if needed).

Step 4. Look at what got generated. You'll see new activities appeared on your canvas without you dragging them individually — this is normal, the wizard builds a small loop-and-extract block for you. Do not delete or rearrange it.

Step 5. Add a numeric column to hold the price as a real number. After the generated block, drag an Add Data Column activity — the scraped Price column is text (e.g. "£15.99"), and filtering by less-than only works on a numeric column, not text.

ColumnName	`"PriceValue"`
DataTable	`bookData`
TypeArgument (Properties → Misc)	`Double`

a) The TypeArgument dropdown is the part that's easy to miss — if it's left as String, the filter step later will fail with a "column type not supported" error.

Step 6. Fill that column inside a loop. Drag a For Each Row activity (see §1.6). Set in to bookData. Inside it, drag an Assign activity.

To	`row("PriceValue")`
Value	`CDbl(row("Price").ToString.Replace("£", "").Trim())`

a) This does two jobs in one line: .Replace("£", "").Trim() strips the currency symbol and stray spaces from the text, and CDbl(...) converts what's left into a real decimal number, written into the new PriceValue column.

Step 7. Filter using a single expression instead of the Filter Data Table wizard. The Filter Data Table wizard reads column types at design time, before your data actually exists — since bookData's schema is only created when the bot runs, the wizard often can't resolve the column type correctly and rejects the < operator. Skip it entirely. Below the loop (not inside it), drag one Assign activity instead.

To	`dtCheapBooks  (Ctrl+K → DataTable)`
Value	`bookData.AsEnumerable().Where(Function(r) r.Field(Of Double)("PriceValue") < 20).CopyToDataTable()`

a) Read it left to right: take every row (AsEnumerable), keep only the ones where PriceValue is under 20 (Where...), then turn the surviving rows back into a proper DataTable (CopyToDataTable).

b) If this line shows a red error, click the Imports tab at the bottom of the screen (next to Variables) and confirm System.Linq and System.Data.DataSetExtensions are both listed — add them if not.

Step 8. Write the results to Excel. Drag a Use Excel File activity below that. In WorkbookPath, click the folder icon and type a brand-new file name, e.g. BookDeals.xlsx — Studio will create this file since it doesn't exist yet.

a) Inside its body, drag Write Range. Set DataTable to dtCheapBooks and Range to "A1".

Step 9. Run and verify. Click Run. Open BookDeals.xlsx afterward and confirm it lists book titles and numeric prices, all under £20.

Expected Output

A new Excel file BookDeals.xlsx listing every book under £20, scraped automatically across multiple pages of books.toscrape.com.

⚠TROUBLESHOOTING

"Column type not supported" from Filter Data Table is exactly why Step 7 bypasses that activity — if you see this error, you're still using the wizard instead of the LINQ expression. If CopyToDataTable() throws an error, it means zero rows matched your filter (e.g. you raised the threshold too high while experimenting) — CopyToDataTable requires at least one row. If Extract Data returns an empty or partial table, the page likely hadn't fully finished loading — re-run the wizard, or increase the generated activities' Delay Before property to 2 seconds.

⍰ EXTENSION CHALLENGE

Change the filter in Step 7 to also keep only highly-rated books (the star rating is stored in a CSS class name on the page, e.g. "star-rating Four" — extract it separately with a Get Attribute activity), or scrape only one category by navigating to that category's URL first.

PROJECT 3

PDF Invoice Extraction Bot

A classic back-office use case: read every PDF in a folder and pull structured fields out using regex (§1.8) and a package you install yourself (§1.7).

Platform	UiPath Studio (Desktop)
Difficulty	Intermediate
Est. Time	55–70 minutes
Prerequisites	UiPath Studio. Sample folder Invoices (5 PDFs: Invoice_1001.pdf – Invoice_1005.pdf) from Workshop_Sample_Files (§1.11).

Learning Outcomes

- Install an extra activity package via Manage Packages.
- Loop over every file in a folder using For Each.
- Extract text from a PDF and pull specific fields out of it using regular expressions.
- Build a results table row by row and flag records that need review.

Step-by-Step Instructions

Step 1. Create a new project. Studio Start screen → Process → name it InvoiceExtractor → Create.

Step 2. Copy the sample folder in. Copy the whole Invoices folder (5 PDF files) from Workshop_Sample_Files into your new project's folder.

Step 3. Install the PDF activities package. Follow §1.7 exactly: Design tab → Manage Packages → make sure All Packages / Official is selected → search UiPath.PDF.Activities → click it → click Install (bottom right of the details pane) → click Save if prompted → wait for the spinner to finish → close the window.

a) Verify it worked: type Read PDF Text into the Activities Panel search box. If it appears in the results, the package is installed.

Step 4. Add a Sequence and create your results table. Drag a Sequence onto the canvas. Before the loop, you need an empty DataTable to fill in as you go. Search for Build Data Table and drag it in.

a) Click it, then click the "Configure Columns..." wizard link shown on the canvas.

b) Add five columns by clicking the + button five times, naming them exactly: InvoiceNo (String), Amount (String), Date (String), FileName (String), Flag (String). Click OK.

c) In the Properties Panel, find Output > DataTable, type dtInvoices, and press Ctrl+K.

Step 5. Add the loop over every PDF file — spelled out click by click. In the Activities Panel search box, type For Each (not "For Each Row" — plain files use the basic For Each, per §1.6). Click and hold the For Each card, drag your mouse onto the canvas below the Build Data Table activity, and release when you see a green horizontal line indicating the drop position.

b) The For Each activity appears on the canvas as a box with the words "For Each" … "in" … and an empty body area below.

c) Click directly on the small editable text field next to "in". Type exactly:

```
Directory.GetFiles("Invoices", "*.pdf")
```

d) This is a built-in .NET function: Directory.GetFiles(folderPath, searchPattern) returns a list of every file path in the given folder matching the pattern — here, every file ending in .pdf inside a folder named Invoices sitting next to your project.

e) Click the small gear/ellipsis icon near the top-right corner of the For Each box to expand its options, and confirm TypeArgument is set to String — if not, click the dropdown, select String (browse for System.String if it's not listed directly).

f) Notice the loop variable is named item by default (top-left of the loop). You may rename it — double-click the name — to invoicePath for clarity. If you rename it, use invoicePath (not item) in every activity you place inside this loop from now on.

Step 6. Inside the loop: extract the text. With the loop's empty body visible, drag a Read PDF Text activity into it.

FileName	`invoicePath`
Text (output)	`pdfText  (type this, press Ctrl+K, choose String)`

Step 7. Extract the invoice number. Below Read PDF Text, still inside the loop, drag an Assign activity.

To	`invoiceNo  (Ctrl+K → String)`
Value	`System.Text.RegularExpressions.Regex.Match(pdfText, "Invoice #:\s*(\S+)").Groups(1).Value`

a) Refer to §1.8 for how this line is built: it searches pdfText for the literal words "Invoice #:", skips any spacing (\s*), then captures the next non-space chunk (\S+) as Group 1, and .Groups(1).Value returns just that captured chunk as text.

Step 8. Extract the date, the same pattern. Add another Assign activity.

To	`invoiceDate  (Ctrl+K → String)`
Value	`System.Text.RegularExpressions.Regex.Match(pdfText, "Date:\s*([\d/]+)").Groups(1).Value`

Step 9. Extract the total amount. Add another Assign activity.

To	`invoiceAmount  (Ctrl+K → String)`
Value	`System.Text.RegularExpressions.Regex.Match(pdfText, "Total:\s*\$([\d,]+\.\d{2})").Groups(1).Value`

a) This pattern looks for "Total:", spacing, a literal $ sign (outside the capturing group, so it isn't included in the result), then captures digits and commas followed by a decimal point and exactly two digits — matching amounts like 12,750.00.

Step 10. Decide the review flag. Drag an If activity into the loop, below the three Assign activities.

Condition	`CDbl(invoiceAmount.Replace(",", "")) > 10000`

a) CDbl converts text to a decimal number (like CInt, but allows decimals). We remove commas first with .Replace(",", "") because CDbl cannot understand "12,750.00" directly — commas must go before conversion.

Step 11. Then branch (needs review). Drag an Assign activity into Then.

To	`invoiceFlag  (Ctrl+K → String)`
Value	`"REVIEW"`

Step 12. Else branch (normal). Drag an Assign activity into Else.

To	`invoiceFlag`
Value	`"OK"`

Step 13. Add this invoice as a new row in your results table. Below the If (still inside the For Each, after it), drag an Add Data Row activity.

DataTable	`dtInvoices`
ArrayRow	`New Object() {invoiceNo, invoiceAmount, invoiceDate, invoicePath, invoiceFlag}`

a) New Object() {…} builds a one-row array in the same left-to-right order as the columns you created in Step 4b (InvoiceNo, Amount, Date, FileName, Flag) — the order must match exactly.

Step 14. Leave the loop and export. Below the entire For Each activity (not inside it — click empty canvas space first to confirm), drag a Use Excel File activity. Set WorkbookPath to a new file name, e.g. InvoiceSummary.xlsx.

a) Inside it, drag Write Range. Set DataTable to dtInvoices and Range to "A1".

Step 15. Run and inspect. Click Run (F5). Open InvoiceSummary.xlsx and confirm all 5 invoices appear with the right numbers and that Invoice_1003 and Invoice_1005 (the two largest amounts) are flagged REVIEW.

Expected Output

InvoiceSummary.xlsx with one row per invoice (InvoiceNo, Amount, Date, FileName, Flag), correctly marking invoices over $10,000 as REVIEW.

> **⚠TROUBLESHOOTING**
> If every Groups(1).Value comes back empty, the regex isn't matching — add a Write Line activity right after Read PDF Text with pdfText as its text, run once, and look at the Output panel to see exactly what text UiPath actually extracted; compare it character-by-character against your regex's literal words. If CDbl throws an error, double-check you removed commas before converting.

> **⍰ EXTENSION CHALLENGE**
> Replace the fixed 10000 threshold with a variable reviewThreshold set once at the top of the workflow, or add a fourth Assign that extracts the Vendor name using the pattern "Vendor:\s*(.+)".

PROJECT 4

Automating a Web Form with Studio Web

Your first automation built entirely in the browser — same concepts as §1.3–1.6, new vocabulary from §1.10.

Platform	UiPath Studio Web (browser-based)
Difficulty	Beginner
Est. Time	40 minutes
Prerequisites	Automation Cloud account, browser only. Sample file Applicants.csv from Workshop_Sample_Files (§1.11). Uses the public practice form at demoqa.com/automation-practice-form.

Learning Outcomes

- Navigate the Studio Web canvas.
- Connect a Google Sheet as a data source.
- Record browser actions and turn them into a repeatable, parameterized flow.
- Build a loop (a "For Each" card) in Studio Web from scratch.

Step-by-Step Instructions

> ⚠TROUBLESHOOTING
> Earlier drafts of this manual described a direct "Upload CSV" data-source card, and a record-first workflow using chips to insert data. A real, working project confirmed both assumptions were wrong: data comes in through a connected Google Sheet, and the typed-into fields use real string.Format expressions (§1.10a), not chips. The steps below reflect the confirmed, working structure.

Step 1. **Turn Applicants.csv into a Google Sheet.** Go to sheets.google.com → Blank spreadsheet. File → Import → Upload → select Applicants.csv → choose Insert new sheet → Import data. Rename the file/sheet tab to Applicants.

Step 2. **Open Studio Web and create a new automation.** Log in at cloud.uipath.com → Studio Web → + New → Automation. Name it FormFillDemo.

Step 3. **Connect your Google account.** Look for a Connections section (left navigation, or offered automatically the first time you add a Google-related card). Click Add Connection → Google Sheets (or Google Drive if Sheets isn't listed separately) → sign in with the account that owns the sheet from Step 1 → approve the permissions.

Step 4. **Add a Read Range (Google Workspace) card.** Click + → search Google Sheets or Read Range → add it.

Connection	`the connection from Step 3`
File	`browse and select Applicants (your Google Sheet)`
Range	`Applicants  – just the sheet/tab name, per §1.10c, not a cell range`
HasHeaders	`On/checked`

a) This card produces a rows output (commonly named readRange) — you'll loop over readRange.Rows.AsEnumerable in the next step.

Step 5. Add a Use Browser card. Click + → search Browser or Use Browser → add it (it may appear as "Use Browser: <app>" once configured).

a) Inside it, add a Navigate Browser card and set its URL to: https://demoqa.com/automation-practice-form

Step 6. Add a For Each card, looping over the sheet's rows. Click + inside the browser card's body → search For Each → add it.

a) Set the In field to the Read Range card's rows output — select it from the field's dropdown/menu rather than typing it. It should resolve to something like readRange.Rows.AsEnumerable.

b) Note the loop variable's name (commonly currentDataRow1) — you'll reference its columns by dot notation below (§1.10c).

Step 7. Add a Type Into card for each field, inside the loop. Click + inside the For Each card's body → search Type Into → add it. Click the target field on the live practice form to anchor the selector (First Name first).

a) In its Text field, click the {} / Expression Editor icon (§1.10a) and type:

```
string.Format("{0}", currentDataRow1.FirstName)
```

b) Add three more Type Into cards the same way — the working project named them Type Into 'Last Name', Type Into 'User Email', and Type Into 'Mobile(10 Digits)', matching the practice form's own field labels — each anchored to its own field and using the matching column: currentDataRow1.LastName, currentDataRow1.Email, currentDataRow1.Mobile.

> **TIP**
> A card called Fill Form exists that can fill an entire form from one row in a single step, instead of one Type Into per field. The working project tried it, then disabled it in favor of four individual Type Into cards — likely for more reliable, individually-anchored selectors. Stick with four Type Into cards unless you specifically want to experiment with Fill Form yourself.

Step 8. Add a Click card for Submit. Click + after the four Type Into cards (still inside the loop) → search Click → add it → anchor it to the form's Submit button.

a) The working project stopped short of this step during testing — treat Submit as the natural next action to add and verify yourself, since it hasn't been confirmed end-to-end the way the rest of this project has.

Step 9. Run it. Click Run (top right). Watch the live execution log — Studio Web shows each card's status (running, succeeded, failed) in real time as it processes each applicant.

Expected Output

The practice form is filled in once for every row in your Applicants Google Sheet — First Name, Last Name, Email, and Mobile — confirmed by each field populating correctly in the run log/screenshots for every iteration.

> **⚠TROUBLESHOOTING**
> If a Type Into card can't find its field on replay, the page layout may have shifted — click that single card and re-anchor its target rather than rebuilding the whole sequence. If the loop only runs once instead of once per row, confirm every card that should repeat is nested inside the For Each's body, not placed after it. If a field's

expression shows the literal text string.Format(…) instead of the actual value once run, the field reverted to plain-text mode — reopen it and confirm Expression Editor is still selected (§1.10a).

EXTENSION CHALLENGE

Add an If card before the Type Into cards (still inside the loop) that checks currentDataRow1.Email.ToString.Contains("@") = False, and if true, skip the rest of the row and log it instead — simulating basic data validation. Also worth adding: a Delay after Submit, and confirmation the "Thanks for submitting the form" message actually appears before moving to the next row.

PROJECT 5

AI-Powered Customer Feedback Analyzer

The first AI-powered exercise: let a language model do the reading and judging, while UiPath does the orchestration. Every field name, variable name, and expression below is copied exactly from a real, working, exported project — not reconstructed from memory.

Platform	UiPath Studio Web + Google Sheets + Google Gemini free API
Difficulty	Intermediate
Est. Time	45–60 minutes
Prerequisites	Studio Web access + a free Google AI Studio API key (steps below). Sample file Reviews.csv from Workshop_Sample_Files, imported into a Google Sheet.

Learning Outcomes

- Understand what is, and isn't, free about this exercise on a Community license.
- Connect Google Sheets as a data source using a Google Workspace card.
- Call a large language model from a UiPath workflow using an HTTP Request card, respecting its rate limit.
- Build a JSON request body reliably using string.Format in a card's Expression Editor (§1.10a).

Step-by-Step Instructions

Before You Start: Is This Actually Free on a Community License?

Short answer: yes, with one clarification. The UiPath Community Edition itself places no restriction on calling an external API — the HTTP Request card used below is a completely standard, free card in Studio Web. What costs money (if anything) is the language model service on the other end of that call, not UiPath.

Option	Cost	Notes
Google AI Studio (Gemini API)	Free tier, no credit card required	Used throughout this project. 5 requests per minute per model on the free tier — see the Delay step below.
OpenAI API (ChatGPT)	Pay-as-you-go, billing required	Needs a funded account; free trial credit availability has changed over time and shouldn't be assumed.

TIP

Option	Cost	Notes

Google's model lineup changes fast — several model names were retired mid-session while this manual was being tested. The confirmed, working project this chapter is based on actually used two different models for its two API calls (below) — a reminder that whichever exact name you use, treat it as disposable and be ready to swap it. If you get a 404 "model no longer available" error, visit https://generativelanguage.googleapis.com/v1beta/models?key=YOUR_API_KEY in a browser for the exact current list — don't guess.

Getting your free API key

Step 1. Go to Google AI Studio. In your browser, visit aistudio.google.com.

Step 2. Sign in. Use any Google account (a personal Gmail account works fine). No credit card or payment method is requested for the free tier.

Step 3. Create an API key. Look for a button labelled Get API key (often top-left or in a side menu). Click it, then click Create API key.

Step 4. Copy and save it. Click the copy icon next to the newly generated key. Paste it somewhere safe temporarily — you'll paste it into a Studio Web card in a moment. Treat this key like a password: don't share it publicly, and don't leave it sitting in a project file you might hand to someone else without removing it first.

WHAT IS "HTTP REQUEST / API CALL"?

A way for one piece of software to ask another piece of software (running somewhere else on the internet) to do something and send back an answer. Here, UiPath sends Google's servers a piece of text and a question, and gets back the model's answer as data.

WHAT IS "JSON"?

A simple, structured way of writing data as text, using curly braces { } for objects and square brackets [ ] for lists, with "key": value pairs inside. APIs almost always send and receive JSON.

Step A — Get your data into Google Sheets

Step 1. Import the CSV. Go to sheets.google.com → Blank spreadsheet → File → Import → Upload → select Reviews.csv → choose Insert new sheet → Import data. Rename the file to Reviews.

Step B — Create the automation and connect your data

Step 2. Create a new automation in Studio Web. Log in at cloud.uipath.com → Studio Web → + New → Automation. Name it FeedbackAnalyzer.

Step 3. Add a Read Range (Google Workspace) card. Click + on the canvas → search Google Sheets or Google Workspace → add the Read Range card. In the exact working project, this card was named Read Range - Read Input Data — a display name is just a label, so use any name that's clear to you.

a) Set up the connection with your personal Google account (approve the permissions prompt).

b) Pick the Reviews file from your Drive.

c) In the Range field, type just the sheet/tab name: Reviews — not a cell range (§1.10c).

d) Turn on HasHeaders.

e) Name this card's output readRange — every expression later in this project refers to that exact name.

Step C — Loop over every review

Step 4. Add a For Each card. Click + → search For Each → add it.

a) Set the In field by selecting readRange's rows output from the menu — pick it from the list rather than typing it. It should resolve to readRange.Rows.AsEnumerable.

b) Keep the loop variable's default name, currentDataRow1 — every expression below uses that exact name. If Studio Web names yours differently, use your actual name consistently instead.

Step D — Call Gemini for the sentiment

Step 5. Add an HTTP Request card inside the loop. Click + inside the For Each card's body → search HTTP Request → add it.

Request Method	`POST`
Request URL	`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent`
Parameters	`key = key  (the parameter NAME must be lowercase "key"), value = your API key`
Headers	`Content-Type = application/json`
Request Body Type	`Text`
Text-Encoding	`UTF-8`
Result (output variable)	`sentimentresponseContent`

⚠TROUBLESHOOTING

If you get a 400 error "Unknown name 'Key': Cannot bind query parameter", your Parameters entry is capitalized as Key instead of key — Google's API is case-sensitive here.

Step 6. Build the request body with string.Format, in Expression Editor mode. Click the small icon at the corner of the Text field (three lines / {}) and choose Expression Editor (§1.10a). Type exactly:

```
string.Format("{{""contents"": [{{""parts"": [{{""text"": ""Classify the sentiment of this review as exactly one word - Positive, Negative, or Neutral. Reply with only that one word. Review: {0} ""}}]}}]}}", currentDataRow1.ReviewText)
```

a) string.Format(template, value) drops currentDataRow1.ReviewText into the {0} placeholder. The doubled {{ }} produce literal single braces, and the doubled "" produce literal double-quotes (§1.4) — together they build valid JSON as the final text.

⚠TROUBLESHOOTING

If you instead get a 400 error "Invalid JSON payload received... Parsing terminated" showing literal doubled quotes in Google's message, the field is still in plain-text mode, not Expression Editor mode. Recheck §1.10a.

TIP

This card also has a Timeout property, hidden under an Advanced section, defaulting to 10 seconds (§1.10b). The confirmed working project left it at that default and it ran fine for this simple prompt — raise it only if you actually hit a "task was canceled" error, rather than pre-emptively.

Step E — Respect the free tier's rate limit

Step 7. Add a Delay card right after the HTTP Request card. Click + directly below the HTTP card (still inside the loop) → search Delay → add it. Set its duration to 20 seconds.

a) The free tier allows 5 requests per minute per model. Since this project makes two calls per review, 20 seconds after each call keeps you safely under that limit.

> **⚠TROUBLESHOOTING**
> A 429 error ("RESOURCE_EXHAUSTED... quota exceeded") means you've hit this limit — the error message itself tells you how many seconds to wait. A 503 ("model is currently experiencing high demand") is a separate, genuine issue on Google's end — just retry after a short wait; nothing in your workflow is wrong when you see this one. Note also that the HTTP Request card already retries automatically up to 3 times on both of these (and other transient errors) before giving up, so brief spikes are often absorbed without you noticing.

Step F — See the raw response (recommended before parsing it)

Step 8. Add a Log Message card right after the HTTP card (before or after the Delay — the working project places it after, right before the Delay). Level: Info. Message (Expression Editor):

```
string.Format("{0}", sentimentresponseContent.TextContent)
```

a) Run once and check the Output panel — you should see the full Gemini JSON response, including a candidates list containing your answer.

Step G — Parse the JSON

Step 9. Add a Deserialize JSON card. JSON String* (Expression Editor):

```
string.Format("{0}", sentimentresponseContent.TextContent)
```

a) Name its output jsonObject1 — used by name in the next step.

Step 10. Add an Assign card to pull out the sentiment. To: create a new String variable named sentiment (lowercase s, matching the working project exactly). Value field: switch to Expression Editor (§1.10a) and type:

Value (Expression Editor)	`jsonObject1("candidates")(0)("content")("parts")(0)("text").ToString().Trim()`

a) Every key here must be lowercase exactly as shown — candidates, content, parts, text — and (0) must be a plain number, not "0" in quotes.

Step 11. Confirm it. Add a Log Message (Level: Info), Message (Expression Editor): string.Format("Sentiment : {0}", sentiment). Run once and confirm it prints a single clean word like Positive or Negative, with nothing else around it.

Step H — Repeat the same pattern for the summary

Step 12. Add a second HTTP Request card, same settings as Step 5 — except two things. The working project used a different model for this second call, and a different prompt:

Request URL	`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent`
Result (output variable)	`summaryResponseContent`

```
string.Format("{{""contents"": [{{""parts"": [{{""text"": ""Summarize this review in under 12 words, plain text only. Review: {0} ""}}]}}]}}", currentDataRow1.ReviewText)
```

a) Add its own Log Message (string.Format("Summary Request - {0}", summaryResponseContent.TextContent)), its own Delay (20 seconds), its own Deserialize JSON (output named jsonObject2), and its own Assign.

To	`Summary` – capital S, matching the working project exactly
Value (Expression Editor)	`jsonObject2("candidates")(0)("content")("parts")(0)("text").ToString().Trim()`

Step 13. Confirm it. Add a Log Message: string.Format("Summarized Text: {0}", Summary).

Where the confirmed, working project stops

> ⚠TROUBLESHOOTING
> The real project this chapter is based on ends exactly here — it logs sentiment and Summary for each review but does not write them anywhere yet. Treat writing the results to a sheet as your own next step, not a gap in your build.

Step 14. Optional — save your results. In Google Sheets, add a tab named FeedbackReport with header row ReviewID, Sentiment, Summary. Add a Write Row (Google Workspace) card after both Assigns, still inside the loop, using the same connection as Step 3, WriteMode: Append — the same card type used (and confirmed working) for writing output in Project 6.

Step 15. Test small, then run fully. Before running on all 10 reviews, consider limiting the loop to the first 2–3 rows to confirm both prompts return clean values. With two 20-second delays per review, a full run takes roughly 7–10 minutes — mention this up front so nobody assumes the bot has frozen.

Expected Output

Every review in your Google Sheet processed with a Sentiment (Positive/Negative/Neutral) and a Summary logged for each — plus, if you complete the optional Step 14, a FeedbackReport sheet with all results saved.

> ⚠TROUBLESHOOTING
> A 404 "model no longer available" error names the exact replacement model to use — swap it directly into the Request URL. A JsonReaderException ("Unexpected character... U") on Deserialize JSON usually means its JSON String input is wired to the wrong output (a whole response object instead of .TextContent) — recheck Step 9. A cancelled/timeout error can mean that specific call is slow to respond (raise the hidden Timeout, §1.10b) or that Google's API is briefly overloaded (a 503 you'll see in the log if you added Step 8's Log Message) — both are worth distinguishing before assuming your workflow is broken.

> ⍰ EXTENSION CHALLENGE
> Add a sentiment-count summary row, or extend the prompt to also extract a specific complaint category (e.g. "Shipping", "Quality", "Price") so results can be routed to different teams.

PROJECT 6

Autonomous Support Ticket Triage Agent

The capstone project: classify each ticket, route it, and — only for the urgent ones — draft a reply for a human to review. Every mechanic here (string.Format bodies, Expression Editor parsing, rate-limit delays) was proven in Project 5.

Platform	UiPath Studio Web + Google Sheets + Google Gemini free API
Difficulty	Advanced (Capstone)
Est. Time	60–75 minutes
Prerequisites	Completion of Project 5 (reuses the same API key, connection, and HTTP pattern). Sample file Tickets.csv from Workshop_Sample_Files, imported into a Google Sheet with extra tabs pre-created (Step 1).

Learning Outcomes

- Classify free-text input into structured categories using a single, simple prompt.
- Use the model's output to drive branching (routing) logic inside a workflow.
- Generate a draft response only where it's actually needed, with a human still checking it before it's sent.
- Recognize a specific, easy-to-make bug — wrapping a real expression in an extra pair of quotes — that silently breaks routing logic.

Step-by-Step Instructions

Step 1. Import Tickets.csv into Google Sheets, and pre-create the queue tabs. sheets.google.com → Blank spreadsheet → File → Import → Upload → Tickets.csv → Insert new sheet → Import data. Rename that tab's sheet file to Tickets.

a) In the same Google Sheets file, add five more empty tabs (click the + at the bottom), named exactly: TechnicalQueue, BillingQueue, AccountQueue, DefaultQueue, EscalationList. The workflow writes into these by name — they must already exist.

Step 2. Create a new automation. Studio Web → + New → Automation → name it TicketTriageAgent.

Step 3. Add a Read Range (Google Workspace) card. Connect to your Google account, browse to your Tickets file.

Range	`Tickets` – just the sheet/tab name (§1.10c)
HasHeaders	`On/checked`
Result (output variable)	`readRange`

Step 4. Add a For Each card. Set In to readRange's rows output, selected from the menu (not typed) — it should resolve to readRange.Rows.AsEnumerable. Keep the loop variable's default name, currentDataRow1.

Classify the ticket in one simple call

Step 5. Add an HTTP Request card inside the loop. Same Request Body Type (Text) and Text-Encoding (UTF-8) as Project 5.

Request Method	POST
Request URL	https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-flash:generateContent
Parameters	key = key (lowercase), value = your API key
Headers	content-type = application/json (lowercase here — the working project used a different capitalization than Project 5's Content-Type; both work, HTTP header names aren't case-sensitive)
Result (output variable)	responseContent

a) Keep the classification prompt plain — don't embed an example JSON schema with escaped quotes inside it. This simpler instruction avoids a whole category of quoting mistakes and is exactly what the working project used:

```
string.Format("{{""contents"": [{{""parts"": [{{""text"": ""Classify this message into Billing or Account or Technical or others and the urgency of the message as low,medium or high. Reply in JSON Format without Code fences or Explanations. Message: {0} ""}}]}}]}}", currentDataRow1.Message)
```

> **TIP**
> This call also has the hidden 10-second Timeout described in §1.10b. The working project left it at the default here too — raise it only if you actually see a "task was canceled" error, since this prompt is more complex than Project 5's and slightly more likely to need it.

Step 6. Add a Log Message right after the HTTP card (before the Delay), printing the raw response. Message (Expression Editor): string.Format("{0}", responseContent.TextContent). Seeing the real response here catches problems early, before they propagate into the parsing steps below.

Step 7. Add a Delay (20 seconds). Same rate-limit reasoning as Project 5.

Step 8. Add a Deserialize JSON card. JSON String* (Expression Editor): string.Format("{0}", responseContent.TextContent). Output: jsonObject.

Step 9. Add an Assign to pull out the model's raw text reply. Expression Editor mode (§1.10a). Note: no .Trim() on this one — the working project omits it here (it's added later, on the final DraftReply instead).

To	rawModelReply (String)
Value (Expression Editor)	jsonObject("candidates")(0)("content")("parts")(0)("text").ToString()

a) rawModelReply is itself text that looks like JSON (e.g. {"category": "Billing", "urgency": "High"}) — because the model always returns its answer as one text field. A second Deserialize JSON, on this inner text, turns it into real, readable data.

Step 10. Add a Log Message printing rawModelReply. Message (Expression Editor): string.Format("{0}", rawModelReply). Confirms what the model actually returned before you try to parse it further.

Step 11. Add a second Deserialize JSON card. JSON String* (Expression Editor): string.Format("{0}", rawModelReply). Output: ticketData.

Step 12. Add two Assign cards to extract the fields — both in Expression Editor mode. This exact step is where a real, live bug was found in a student's project — read the warning below before typing anything.

> **⚠TROUBLESHOOTING**

The bug to avoid: typing the whole expression wrapped in an extra pair of quotes, like "ticketData(""category"").ToString()" — this silently turns working code into a literal piece of text, so Category permanently holds the words ticketData("category").ToString() instead of the real value, and every ticket then falls through to your Switch's Default branch no matter what. Type ONLY the code below, with no quotes wrapped around the whole thing.

To	`Category  (String)`
Value (Expression Editor — no outer quotes)	`ticketData("category").ToString()`
To	`Urgency  (String)`
Value (Expression Editor — no outer quotes)	`ticketData("urgency").ToString()`

Step 13. Add a Log Message printing both. Message (Expression Editor): string.Format("Category - {0} and Urgency - {1}", Category, Urgency) — confirms both fields extracted real values before routing on them.

Route by category

Step 14. Add a Switch card. Expression (Expression Editor): string.Format("{0}", Category). Add branches for exactly: Technical, Billing, Account, and use the Switch's Default for anything else.

a) In each branch (including Default), add a Write Row (Google Workspace) card: DataRow = currentDataRow1, DataType = DataRow, WriteMode = Append, HasHeaders = Off, pointing at the matching tab from Step 1a — TechnicalQueue, BillingQueue, AccountQueue, or DefaultQueue for the Default branch.

⚠TROUBLESHOOTING
If every ticket lands in Default no matter what, this is almost always the Step 12 quoting bug above, not a Switch misconfiguration — check Category's actual value with the Step 13 log before changing anything else.

Escalate and draft a reply — only for urgent tickets

Step 15. Add an If card after the Switch, still inside the loop. Condition (Expression Editor): Urgency = "High"

a) In Then, add a Write Row card appending the ticket to EscalationList — same field pattern as Step 14a.

Step 16. Still inside Then, add a second HTTP Request card to draft a reply. Same Method/Parameters/Headers/Request Body Type as Step 5.

Request URL	`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-flash:generateContent`
Result (output variable)	`responseContent_DraftReply`

```
string.Format("{{""contents"": [{{""parts"": [{{""text"": ""Write a short, polite draft reply (3-4 sentences) to this customer message, acknowledging their issue. Do not promise a specific resolution time. Message: {0}""}}]}}]}}", currentDataRow1.Message)
```

Step 17. Add a Delay (20s), a Deserialize JSON, and an Assign — same pattern as Steps 7–9. Deserialize JSON String* (Expression Editor): string.Format("{0}", responseContent_DraftReply.TextContent). Output: jsonObject_DraftReply.

To	`DraftReply (String)`
Value (Expression Editor)	`jsonObject_DraftReply("candidates")(0)("content")("parts")(0)("text").ToString().Trim()`

Step 18. Add a Log Message printing DraftReply. Message (Expression Editor): string.Format("Drafted Message : {0}", DraftReply). Nothing here sends the reply anywhere — that's the point of the discussion in Step 19.

Wrap up

Step 19. Run, inspect, and discuss. Run the flow on the sample tickets — non-urgent tickets take one API call each (~20+ seconds), urgent ones take two. As a group: which classifications were correct? Would you trust this bot to auto-send the draft reply, or should a person approve it first? That question is the actual point of the capstone, not just getting the workflow to run.

Expected Output

Tickets routed into their matching queue tab in Google Sheets (TechnicalQueue, BillingQueue, AccountQueue, or DefaultQueue), with high-urgency tickets additionally appended to EscalationList alongside an AI-drafted reply ready for a person to review.

⚠TROUBLESHOOTING

The same errors and fixes from Project 5 apply here (400 on the API key parameter case, 400 on malformed JSON from typing code into a plain-text field, 404 model retirement, 429 rate limit, and the hidden 10-second Timeout from §1.10b causing "A task was canceled") — see Project 5's Troubleshooting box for each in detail. Specific to this project: if the model wraps its classification JSON in extra commentary (e.g. "Sure, here you go: {...}"), tighten the prompt ("reply with raw JSON only, no commentary, no markdown code fences"). If Category or Urgency come back looking like literal code text instead of a real value, see the Step 12 warning above — that exact bug was found in a real, live project during this manual's development.

⍰ EXTENSION CHALLENGE

Add a human-in-the-loop approval step (an Action Task / approval card, if your Studio Web plan offers one) so a person approves or edits each draft reply before it's marked as sent. Also consider generating a draft reply for every ticket, not just urgent ones, once the basic flow is solid — the real project this manual is based on deliberately kept it to urgent-only first, to prove the pattern before extending it.

Appendix A: UiPath Activity Cheat Sheet

Activity	What it does	Used in
Read Range / Write Range	Read or write a block of Excel data as a DataTable	1, 2, 3
For Each Row	Loop through rows of a DataTable	1, 2
For Each	Loop through a plain list (files, or Studio Web rows)	3, 4, 5, 6
Extract Data (Data Scraping)	Pull a repeating structure (table/list) off a webpage	2
Read PDF Text	Extract raw text from a PDF file	3
Regex.Match / Regex.Matches	Pull a pattern (e.g. invoice number) out of text	3
Browser card / Use Application-Browser	Open and interact with a website	2, 4
Read Range (Google Workspace)	Read rows from a connected Google Sheet	4, 5, 6
Add Row (Google Workspace)	Append a new row to a connected Google Sheet	5, 6
HTTP Request	Call any web API, including an LLM's chat completions endpoint	5, 6
Deserialize JSON	Turn a JSON string into a queryable object	5, 6
Switch	Multi-branch routing based on a value	6
Human-in-the-loop / Action Task	Pause a flow for a person to approve or edit before continuing	6 (extension)

Appendix B: GPT Prompt Templates for RPA

These are starting points, not final answers — tightening a prompt is often the fastest fix when a bot's AI step misbehaves.

Classification (single label)

```
Classify the following text as exactly one of: {label1, label2, label3}.
Reply with only the label, no punctuation or explanation. Text: {{input}}
```

Structured extraction (JSON)

```
Extract the following fields from the text and reply with raw JSON only,
no commentary, no markdown code fences: {"field1":"","field2":""}. Text: {{input}}
```

Summarization

```
Summarize the following in under {{n}} words, plain language, no preamble: {{input}}
```

Draft reply generation

```
Write a short, polite reply (3-4 sentences) to this message. Do not make
promises about timing or outcomes. Message: {{input}}
```

TIP

Keep the surrounding JSON scaffolding ({"contents": [{"parts": [{"text": "..."}]}]}) as plain literal text in a Studio Web body field, and insert only the {{input}} part as a chip — see §1.10a. Don't type VB.NET-style quote-escaping into these fields; that pattern is for desktop Studio expression fields only.

Appendix C: Glossary

Term	Meaning
RPA	Robotic Process Automation — software that mimics human actions on a computer (clicking, typing, reading screens/files) to complete rule-based tasks.
Selector	The UiPath-generated identifier used to find a specific UI element reliably across runs.
Workflow	A sequence, flowchart, or state machine file (.xaml) representing one automation.
Orchestrator	UiPath's central server/cloud service for deploying, scheduling, and monitoring bots at scale.
Studio Web	UiPath's browser-based, no-install automation builder, aimed at faster and simpler flows than desktop Studio.
Agentic automation	Automation where an AI model contributes judgment or reasoning (e.g. classification, drafting) as one step inside an otherwise rule-based RPA flow.
Expression Editor	A mode some Studio Web fields can be switched into (via a small icon at the field's corner) that accepts real VB.NET code — indexers, concatenation, function calls — instead of plain text with chips. See §1.10a.

Appendix D: A Note on API Details

The exact API endpoint names, model names (e.g. gemini-3.5-flash), and UiPath card names referenced in this manual can change as vendors update their products — this happened three times in the space of one afternoon while this manual's Projects 5 and 6 were being tested live. Before running the workshop, the facilitator should do a quick dry run of Projects 5 and 6, and if a model name has been retired, visit https://generativelanguage.googleapis.com/v1beta/models?key=YOUR_API_KEY to get the current exact list rather than guessing a replacement.

Appendix E: Where to Find the Sample Files

This manual is distributed together with a Workshop_Sample_Files folder (or .zip) containing every dataset referenced above. If you received this manual without that folder, ask your facilitator for it before starting Project 1 — none of the projects will work without their corresponding sample file, except Project 2, which uses a live public website instead of a local file. For Projects 4, 5, and 6, that sample file gets imported into Google Sheets rather than uploaded directly — see each project's setup steps.